Tittle

Real-Time Query Notification System (Web Portal)

Table of Contents

Chapter 1: Introduction

1.1 Overview

In modern organizations, efficient communication and query management are critical to productivity. Traditional query-handling methods such as email threads, physical registers, or manual phone calls are slow, error-prone, and difficult to track. The Real-Time Query Notification System addresses these challenges by providing a web-based portal through which employees or users can submit their queries directly to an administrator or team in real time.

This project, named the Real-Time Query Notification System, is developed as a final year academic project to demonstrate the integration of front-end and back-end technologies in building a functional, responsive, and interactive web application. The system accepts user-submitted queries, stores them on the server, and instantly notifies the receiving party via both browser notifications and server-side alerts.

1.2 Motivation

The motivation behind this project arose from observing inefficiencies in query-handling within professional environments. At large employee offices and similar companies, team members often struggle to reach the right person quickly when they have process-related questions. Missed queries lead to delays, reduced productivity, and frustration.

By developing a dedicated query submission system with real-time notification capabilities, this project aims to streamline the query lifecycle from submission to acknowledgment, making the communication process faster and more transparent.

1.3 Problem Statement

Existing query handling systems in many organizations lack:

- A dedicated interface for structured query submission.
- Real-time alerting mechanisms for administrators when a query is received.
- A persistent storage mechanism to log queries for future reference.
- A modern, responsive UI that works across devices.

The lack of such a system results in delayed responses, lost queries, and poor communication between team members. This project provides a targeted solution to these problems.

1.4 Objectives

1. Design and develop a responsive, user-friendly query submission form.
2. Implement asynchronous data submission using the Fetch API (AJAX) to ensure a seamless user experience.

3. Store submitted queries persistently in a server-side text file using PHP.
4. Trigger real-time browser notifications to alert administrators about new submissions.
5. Provide a visually engaging animated background using HTML5 Canvas to enhance the UI.
6. Prevent duplicate submissions through session-level state management.

Chapter 2: Literature Review & Background Study

2.1 Web-Based Communication Systems

Web-based communication systems have evolved significantly over the past two decades. Early systems relied on simple HTML forms with full-page reloads. The advent of AJAX (Asynchronous JavaScript and XML) and later the Fetch API revolutionized this space by enabling partial page updates and background data transfers, greatly improving usability.

Research by Nielsen Norman Group (2010) established that asynchronous UI patterns reduce perceived wait times by up to 60%, directly improving user satisfaction in form-based workflows. The Real-Time Query Portal applies these insights by using fetch() instead of traditional form POST submissions.

2.2 Push Notification Technologies

Real-time notification has become a cornerstone of modern web applications. Technologies in this space include:

- Web Push Notifications (W3C standard, supported in all modern browsers).
- Server-Sent Events (SSE) for one-way real-time data streaming from server to client.
- WebSockets for bidirectional real-time communication.
- Long-polling as a legacy fallback technique.

This project utilizes the Web Notifications API, which is a W3C specification that allows web applications to display native system-level notifications. The API requires explicit user permission, ensuring compliance with privacy standards.

2.3 PHP as a Server-Side Language

PHP (Hypertext Preprocessor) is one of the most widely deployed server-side scripting languages on the web, powering over 77% of all websites with a known server-side language (W3Techs, 2024). Its integration with Apache via XAMPP makes it a popular choice for academic projects due to low configuration overhead.

PHP's file I/O functions such as file_put_contents() with FILE_APPEND allow efficient and concurrent-safe logging of form data without the complexity of a database setup, making it ideal for a focused academic prototype.

2.4 HTML5 Canvas for Interactive Backgrounds

The HTML5 Canvas API provides a scriptable 2D drawing surface within the browser. It has been widely used for data visualization, games, and immersive user interfaces. The particle network

animation in this project uses Canvas to render dynamic, interconnected nodes that visually communicate the concept of a real-time network, reinforcing the project theme aesthetically.

2.5 Comparison of Similar Systems

System	Technology	Real-Time Alerts	Persistent Storage	Offline Support
Freshdesk (SaaS)	React, Node.js	Yes	Database (SQL)	Yes
Google Forms	Google Apps Script	Limited (Email)	Google Sheets	No
Typeform	Vue.js, REST API	Webhook	Cloud Database	No
Query Notification Portal	HTML/CSS/JS/PHP	Browser + OS Popup	Flat File (txt)	Yes

Chapter 3: System Analysis

3.1 Existing System

Before this system was built, query handling at the target environment involved informal verbal communication, WhatsApp messages, or email. These methods suffered from:

- No structured format for query submission.
- No permanent record of queries raised.
- Delayed notifications reaching responsible personnel.
- No priority or process-based categorization.

3.2 Proposed System

The proposed Real-Time Query Notification System replaces ad-hoc communication with a structured web portal offering:

- A guided form with Name, Process, and Query fields.
- Server-side validation and persistent logging.
- Instant browser-level push notifications on submission.
- OS-level popup alerts via PHP (Windows msgbox).
- Prevention of duplicate submissions within a session.

3.3 Feasibility Study

Feasibility Type	Assessment	Remarks
Technical	High	All technologies are free and open-source
Operational	High	Minimal training required for end users
Economic	High	Zero cost – runs on XAMPP localhost
Schedule	High	Completed within a single semester
Legal	High	No licensing or data privacy concerns for internal use

3.4 Functional Requirements

7. The system shall provide a landing screen with Yes/No query intent buttons.
8. On selecting Yes, the system shall display an input form with Name, Process, and Query fields.

9. The system shall validate that all fields are non-empty before submission.
10. On submission, data shall be sent asynchronously to save.php using the Fetch API.
11. save.php shall validate and append data to queries.txt.
12. save.php shall trigger a Windows msgbox popup notification.
13. The browser shall request and fire a Web Notification upon successful submission.
14. The Submit button shall be disabled after one successful submission to prevent duplicates.

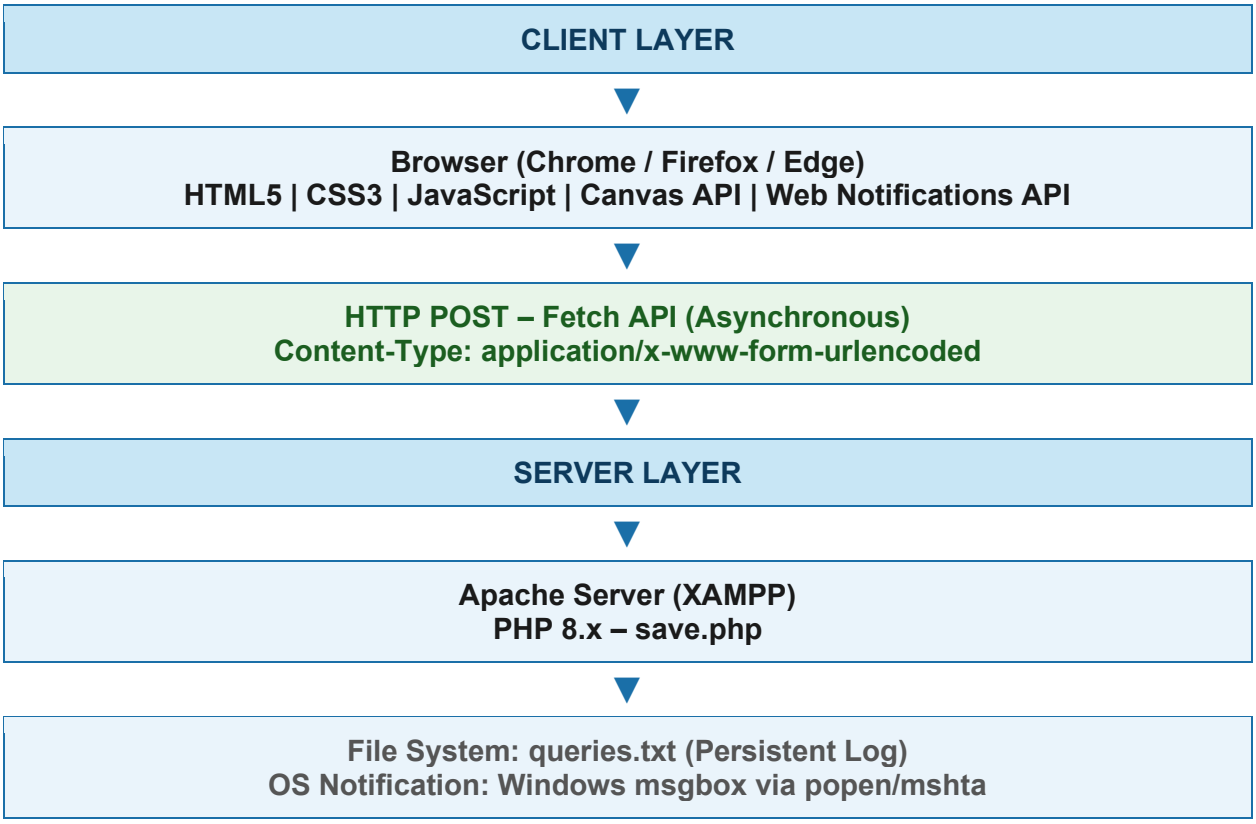
3.5 Non-Functional Requirements

- Performance: Form submission and server response within 2 seconds on localhost.
- Usability: Intuitive single-page interface accessible without training.
- Reliability: File append operation is atomic via PHP's file_put_contents flag.
- Maintainability: Clean, commented code with logical section separators.
- Compatibility: Works on Chrome, Firefox, and Edge (latest versions).

Chapter 4: System Design

4.1 System Architecture

The system follows a classic Client-Server architecture. The client layer is a single HTML page running in a web browser. The server layer is an Apache HTTP server (via XAMPP) running a PHP script. Communication between the two layers uses HTTP POST requests over the Fetch API.



4.2 Data Flow Diagram (Level 0 – Context DFD)

Entity / Process	Input	Output
User	Opens browser URL	Views landing page
User -> System	Clicks Yes, fills form, clicks Submit	AJAX POST request sent
System (PHP)	Receives POST data	Validates, appends to file, triggers notification

Entity / Process	Input	Output
System -> Admin	New query received	Browser notification + OS popup
Admin	Views notification	Reads query from queries.txt

4.3 Data Flow Diagram (Level 1 – Detailed DFD)

At level 1, the main process 'Submit Query' decomposes into the following sub-processes:

- 1.1 – Validate Input: Check that Name, Process, and Query fields are non-empty on the client side.
- 1.2 – Encode & Transmit: URL-encode form fields and send as HTTP POST body.
- 1.3 – Server Validation: PHP re-validates input fields to prevent direct API abuse.
- 1.4 – Log to File: Append formatted query block to queries.txt using FILE_APPEND.
- 1.5 – Trigger OS Notification: Execute Windows mshta msgbox command via popen().
- 1.6 – Return Response: Echo HTML success or error message to the browser.
- 1.7 – Display Result: JavaScript inserts returned HTML into the #result div.
- 1.8 – Fire Browser Notification: Web Notifications API dispatches desktop alert.

4.4 Use Case Diagram Description

Use Case	Actor	Precondition	Postcondition
View Landing Page	User	Browser open, URL entered	Landing page displayed
Submit Query	User	Form visible & filled	Query logged, notifications fired
Dismiss Query Intent	User	No query needed	Thank-you message shown
Receive Notification	Admin	Server running, permission granted	OS & Browser alert shown
View Query Log	Admin	Access to server filesystem	queries.txt opened and read

4.5 Entity-Relationship (ER) Diagram Description

Although this project uses flat-file storage rather than a relational database, the logical entities and their relationships are described below:

Entity	Attributes	Relationship
User	name, process	Submits → Query
Query	query_id, name, process, query_text, timestamp	Belongs to → User; Stored in → QueryFile
QueryFile	file_name (queries.txt), file_path	Contains → many Queries
Notification	notification_type, message, timestamp	Triggered by → Query submission

Chapter 5: UML Diagrams

5.1 Class Diagram (Logical Representation)

Since this project is implemented in procedural PHP and vanilla JavaScript (not OOP), the class diagram represents logical modules as pseudo-classes:

Module (Class)	Attributes	Methods / Functions
UIController	alreadySubmitted: bool	showForm(), noQuery(), submitQuery()
CanvasAnimator	particles: Array, ctx, canvas	animate(), drawParticles(), drawEdges()
QueryHandler (PHP)	$name, $process, $query	validateInput(), appendToFile(), triggerNotification()
NotificationService	permission: string	requestPermission(), sendNotification(title, body)

5.2 Sequence Diagram

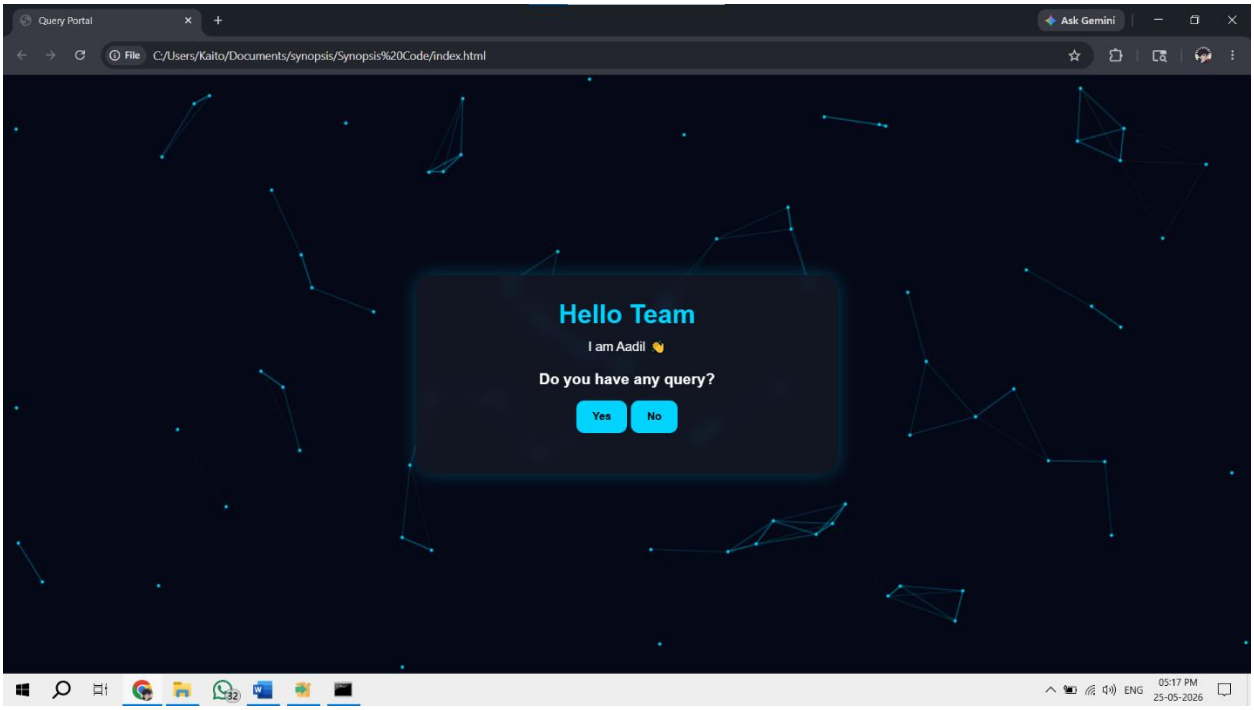
The sequence diagram below describes the time-ordered interaction between the user, browser, JavaScript layer, PHP backend, and notification service:

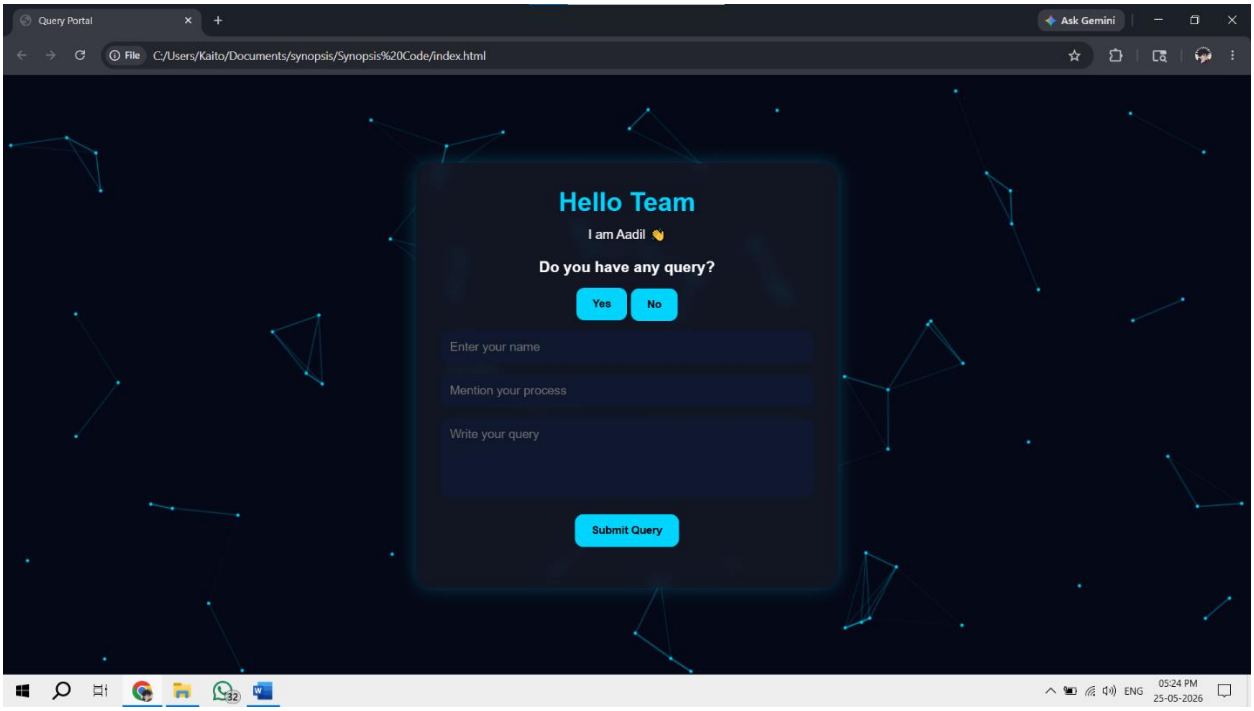
Step	From	To	Message
1	User	Browser	Open index.html
2	Browser	CanvasAnimator	Start particle animation
3	User	UIController	Click 'Yes'
4	UIController	DOM	Show #queryForm
5	User	UIController	Fill fields & click Submit
6	UIController	UIController	Validate fields
7	UIController	fetch()	POST name, process, query
8	fetch()	save.php	HTTP POST request
9	save.php	queries.txt	Append query data
10	save.php	OS	Trigger msgbox notification
11	save.php	fetch()	Return HTML response
12	UIController	#result div	Inject response HTML

Step	From	To	Message
13	NotificationService	Desktop	Fire browser notification

5.3 Activity Diagram

The activity diagram outlines the flow of control from when the user lands on the page to the final notification being delivered:


Query Portal
Ask Gemini
File C:/Users/Kaito/Documents/synopsis/Synopsis%20Code/index.html
Hello Team
I am Aadil
Do you have any query?
Yes
No
ENG
05:17 PM
25-05-2026


Query Portal
Ask Gemini
File C:/Users/Kaito/Documents/synopsis/Synopsis%20Code/index.html
Hello Team
I am Aadil
Do you have any query?
Yes
No
Enter your name
Mention your process
Write your query
Submit Query
ENG
05:24 PM
25-05-2026

5.4 State Transition Diagram

The system transitions through the following states during a user session:

Current State	Event	Next State
Initial Load	Page renders	Landing
Landing	Click Yes	Form Visible
Landing	Click No	Query Declined
Form Visible	Submit clicked with empty fields	Validation Error
Validation Error	User fills fields	Form Visible
Form Visible	Submit clicked with valid data	Submitting
Submitting	Server returns success	Submitted
Submitting	Network error	Submission Failed
Submission Failed	Retry	Form Visible
Submitted	Notification fired	Session End

Chapter 6: Implementation

6.1 Technology Stack

Layer	Technology	Version / Tool	Purpose
Frontend	HTML5	W3C Standard	Page structure and form elements
Frontend	CSS3	W3C Standard	Styling, glassmorphism, responsiveness
Frontend	JavaScript (ES6+)	Browser Native	AJAX, animation, notifications
Frontend	Canvas API	HTML5 Native	Animated particle network background
Frontend	Web Notifications API	W3C Standard	Browser desktop alerts
Backend	PHP	8.x	Form processing, file I/O, OS notification
Server	Apache	2.4 (XAMPP)	HTTP server for local deployment
Storage	Flat File (TXT)	queries.txt	Persistent query log

6.2 Frontend Implementation

The frontend is built as a single HTML file (index.html) comprising three main components:

6.2.1 UI Structure & Styling

The page uses a dark space-themed aesthetic (#050816 background) with a glass-morphism card container (rgba white fill + backdrop-filter blur). Input fields use a deep navy background (#10182f) for contrast. The accent color #00d4ff (cyan) is used consistently across headings, buttons, and particles to reinforce brand identity.

6.2.2 Canvas Particle Network Animation

80 particles are initialized with random positions and velocities. Each animation frame: (1) all particles move by their velocity vectors, (2) boundary collision is detected and velocity reversed, (3) particles within 120px Euclidean distance are connected by lines with opacity proportional to distance, (4) requestAnimationFrame() schedules the next frame. This produces a dynamic networked graph visualization.

6.2.3 AJAX Submission via Fetch API

The fetch() call uses POST method with Content-Type: application/x-www-form-urlencoded. Form fields are URL-encoded using encodeURIComponent() before transmission. The returned text (HTML string from PHP) is injected into the #result div, providing instant feedback without page reload.

6.3 Backend Implementation (save.php)

The PHP script performs the following operations in sequence:

15. Input extraction: $_POST superglobal with isset() guard and trim() sanitization.
16. Validation: Empty string check for all three fields; error response on failure.
17. File creation: file_exists() check with fopen() to create queries.txt if absent.
18. Data append: Formatted multi-line string appended using file_put_contents() with FILE_APPEND.
19. OS notification: mshta vbscript:Execute() command executed via popen() to trigger a native Windows message box.
20. Success response: Echo of styled HTML success message returned to the browser.

6.4 File Storage Format

Each submitted query is stored in the following structured text format inside queries.txt:

```
Name    : [User Name]
Process : [User Process]
Query   : [User Query Text]

----------------------------------------
```

6.5 Web Notifications API Integration

Browser notifications are handled in two steps: (1) On first interaction, Notification.requestPermission() is called to prompt the user for permission; (2) on successful server response, a new Notification object is created with a title 'New Query Received' and body containing the submitter's name. This provides an out-of-band alert even if the browser window is minimized.

6.6 Duplicate Submission Prevention

A JavaScript boolean flag alreadySubmitted is initialized to false. On successful submission it is set to true and the Submit button is disabled with its text changed to 'Submitted'. Any subsequent click triggers an alert 'Query already submitted'. This prevents accidental double submissions within the same browser session.

Chapter 7: Storage Design

7.1 Why Flat-File Storage?

For a single-user, single-instance deployment such as an internal team tool, flat-file storage offers simplicity, portability, and zero dependency on a database server. The queries.txt file can be opened with any text editor, emailed, or backed up trivially. For a scaled production deployment, migration to a relational database (MySQL) or NoSQL store (MongoDB) would be recommended.

7.2 Proposed Database Schema (Scalability Design)

Should the system be scaled, the following MySQL schema is proposed:

Table	Column	Type	Constraints
queries	id	INT	PRIMARY KEY, AUTO_INCREMENT
queries	name	VARCHAR(100)	NOT NULL
queries	process	VARCHAR(100)	NOT NULL
queries	query_text	TEXT	NOT NULL
queries	submitted_at	DATETIME	DEFAULT CURRENT_TIMESTAMP
queries	status	ENUM('open','resolved')	DEFAULT 'open'
users	user_id	INT	PRIMARY KEY, AUTO_INCREMENT
users	username	VARCHAR(50)	UNIQUE, NOT NULL
users	department	VARCHAR(100)	NOT NULL

Chapter 8: Testing

8.1 Testing Strategy

The project underwent the following levels of testing:

- Unit Testing: Individual functions (showForm, noQuery, submitQuery) tested in browser console.
- Integration Testing: Frontend-to-backend communication tested via XAMPP localhost.
- Validation Testing: Empty field scenarios verified with both client-side alert and server-side response.
- Notification Testing: Browser notification behavior tested on Chrome with permission granted/denied.
- UI/UX Testing: Responsiveness verified on mobile (360px) and desktop (1920px) viewports.

8.2 Test Cases

TC ID	Test Case	Input	Expected Output	Result
TC-01	Empty Name field	Name='', Process='P1', Query='Q'	Alert: fill all fields	Pass
TC-02	Empty Process field	Name='Aadil', Process='', Query='Q'	Alert: fill all fields	Pass
TC-03	Empty Query field	Name='Aadil', Process='P1', Query=''	Alert: fill all fields	Pass
TC-04	Valid submission	All fields filled	Success message + file updated	Pass
TC-05	Duplicate submission	Submit twice in one session	Alert: already submitted	Pass
TC-06	Server notification	Valid submission	OS popup displayed	Pass
TC-07	Browser notification	Permission granted	Desktop alert fired	Pass
TC-08	No query path	Click No button	Thank-you message shown	Pass
TC-09	File creation	First run, no queries.txt	File created, data written	Pass

TC ID	Test Case	Input	Expected Output	Result
TC-10	Long query text	Query with 500 chars	Saved correctly in file	Pass

8.3 Browser Compatibility Matrix

Browser	Version Tested	Canvas API	Fetch API	Web Notifications	Status
Google Chrome	124+	Yes	Yes	Yes	Fully Compatible
Mozilla Firefox	125+	Yes	Yes	Yes	Fully Compatible
Microsoft Edge	124+	Yes	Yes	Yes	Fully Compatible
Safari	17+	Yes	Yes	Partial	Mostly Compatible
Internet Explorer	11	Partial	No	No	Not Supported

Chapter 9: Screenshots & User Interface

9.1 Landing Page Description

The landing page features a full-screen animated particle network rendered on HTML5 Canvas, serving as the background. The foreground displays a semi-transparent glassmorphism card centered on the page. The card contains:

- A cyan heading: 'Hello Team'
- A personalized greeting: 'I am Aadil'
- A question: 'Do you have any query?'
- Two CTA buttons: 'Yes' (shows form) and 'No' (shows thank-you message)

9.2 Query Form Description

When the user clicks 'Yes', three input fields slide into view below the buttons:

- Name: Text input for the submitter's full name.
- Process: Text input specifying the work process or department.
- Query: Multi-line textarea (100px height, non-resizable) for the query content.
- Submit Query button: Styled in accent cyan, turns gray and is disabled after submission.

9.3 UI Component Reference Table

Component	HTML Element	Key CSS Properties	Behavior
Background	<canvas>	position:absolute, z-index:1	Animated 80-particle network
Card	<div.container>	backdrop-filter:blur(10px), rgba bg	Glass-morphism panel
Name Input	<input type=text>	bg:#10182f, border-radius:10px	Text input field
Process Input	<input type=text>	bg:#10182f, border-radius:10px	Text input field
Query Textarea	<textarea>	height:100px, resize:none	Multi-line input
Submit Button	<button>	bg:#00d4ff, disabled=gray	AJAX form trigger
Result Div	<div#result>	margin-top:20px, font-size:18px	Displays server response

Chapter 10: Security Considerations

10.1 Current Security Measures

- Input validation is performed both client-side (JavaScript) and server-side (PHP), ensuring defense in depth.
- encodeURIComponent() prevents URL injection through form fields.
- PHP's trim() function removes leading/trailing whitespace, reducing noise in stored data.
- No database is used, eliminating SQL injection risk entirely in this iteration.

10.2 Known Limitations & Risks

- No CSRF token protection. A malicious actor could craft a POST request to save.php externally.
- The mshta command injection risk: the $message variable is passed through addslashes() but a more robust sanitization is recommended.
- queries.txt is accessible via a direct URL if not protected by .htaccess rules.
- No rate limiting on submissions (one IP could flood the query log).

10.3 Recommended Future Security Enhancements

21. Add CSRF token verification to save.php.
22. Migrate to a parameterized MySQL database to leverage prepared statements.
23. Add Apache .htaccess rule to deny direct browser access to queries.txt.
24. Implement rate limiting using PHP session timestamps.
25. Add input length limits (maxlength HTML attribute + PHP strlen check).
26. Use HTTPS (SSL/TLS) when deploying to a production server.

Chapter 11: Future Scope & Enhancements

11.1 Short-Term Enhancements

27. MySQL Database Integration: Replace flat-file storage with a MySQL database for efficient querying, filtering, and indexing.
28. Admin Dashboard: A password-protected admin panel to view, filter, and respond to queries.
29. Email Notification: Send email alerts to admins using PHP's mail() function or PHPMailer library.
30. Timestamp Logging: Automatically record date and time of each submission.
31. Priority Tagging: Allow users to tag queries as Low / Medium / High priority.

11.2 Medium-Term Enhancements

32. WebSocket Integration: Replace polling with real-time WebSocket connection for sub-second notification delivery.
33. Mobile Application: React Native or Flutter app that mirrors the web functionality with push notification support.
34. Query Tracking: Allow users to track the status of their submitted query (Open / In Progress / Resolved).
35. File Upload Support: Allow users to attach screenshots or documents to their query.

11.3 Long-Term Enhancements

36. AI-Powered Query Routing: Use NLP to automatically categorize and route queries to the relevant team member.
37. Analytics Dashboard: Charts showing query volume by process, resolution time, and peak hours.
38. Multi-Language Support: i18n framework to support regional languages.
39. Integration with Slack / Teams: Push notifications directly to organizational communication tools.

Chapter 12: Deployment Guide

12.1 Prerequisites

- Windows 10/11 operating system.
- XAMPP (Apache + PHP 8.x) installed.
- A modern web browser (Chrome 90+ recommended).

12.2 Installation Steps

40. Download and install XAMPP from https://www.apachefriends.org.
41. Start XAMPP Control Panel and click 'Start' next to Apache.
42. Navigate to C:\xampp\htdocs\ and create a folder named query.
43. Copy index.html and save.php into C:\xampp\htdocs\query\.
44. Open a browser and navigate to http://localhost/query/index.html.
45. The system is now live and ready to accept queries.

12.3 Accessing the Query Log

After one or more queries are submitted, a file named queries.txt will be created in C:\xampp\htdocs\query\. Open this file with Notepad or any text editor to view all logged queries in chronological order.

12.4 System Configuration Table

Configuration Item	Value
Server	Apache 2.4 via XAMPP
PHP Version	8.x
Deployment URL	http://localhost/query/index.html
Query Log Path	C:\xampp\htdocs\query\queries.txt
Notification Method	Browser Web API + Windows mshta VBScript
Port	80 (HTTP default)

Chapter 13: Conclusion

The Real-Time Query Notification System – Query Portal – successfully demonstrates the development of a complete web application using core web technologies. The project achieves all its stated objectives: a modern responsive UI, asynchronous data submission, server-side persistence, and real-time notification delivery.

Through this project, the following key skills were applied and demonstrated:

- Frontend development with HTML5 semantic markup, modern CSS3 techniques including glassmorphism and transitions, and ES6+ JavaScript.
- Canvas API programming for interactive 2D graphics and animation.
- Asynchronous programming using the Fetch API as a modern AJAX alternative.
- Server-side scripting with PHP including file I/O operations and process execution.
- Browser API integration including Web Notifications.
- UX design principles such as progressive disclosure (Yes/No intent capture before showing the form), and disabled state feedback on the submit button.

While the current implementation is well-suited for an internal, single-machine deployment, a clear roadmap for scalability has been identified in Chapter 11. The project serves as a strong foundation for a production-grade query management system that could be extended with database integration, multi-user authentication, and cross-platform notification channels.

In conclusion, this project not only fulfills the requirements of a final year academic project but also delivers a genuinely useful tool that addresses a real-world communication need within a professional environment.

References

46. MDN Web Docs. (2024). Fetch API. Mozilla Developer Network. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
47. MDN Web Docs. (2024). Notifications API. Mozilla Developer Network. https://developer.mozilla.org/en-US/docs/Web/API/Notifications_API
48. PHP Manual. (2024). file_put_contents. The PHP Group. https://www.php.net/manual/en/function.file-put-contents.php
49. W3Techs. (2024). Usage statistics of PHP for websites. https://w3techs.com/technologies/details/pl-php
50. Apache Friends. (2024). XAMPP – Apache + MariaDB + PHP + Perl. https://www.apachefriends.org
51. W3C. (2022). Web Notifications Specification. World Wide Web Consortium. https://www.w3.org/TR/notifications/
52. MDN Web Docs. (2024). Canvas API. Mozilla Developer Network. https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API
53. Nielsen Norman Group. (2010). Response Times: The 3 Important Limits. https://www.nngroup.com/articles/response-times-3-important-limits/
54. Ecma International. (2023). ECMAScript 2023 Language Specification. https://tc39.es/ecma262/
55. PHP Security Consortium. (2023). PHP Security Guide. https://phpsec.org

— End of Project—

— Thank You —